*** Insert the following elements in the Hastable data structure using hashing function :**

22, 93, 31, 88, 56, 97

Bucket index = key % table length

22 % 10 = 2

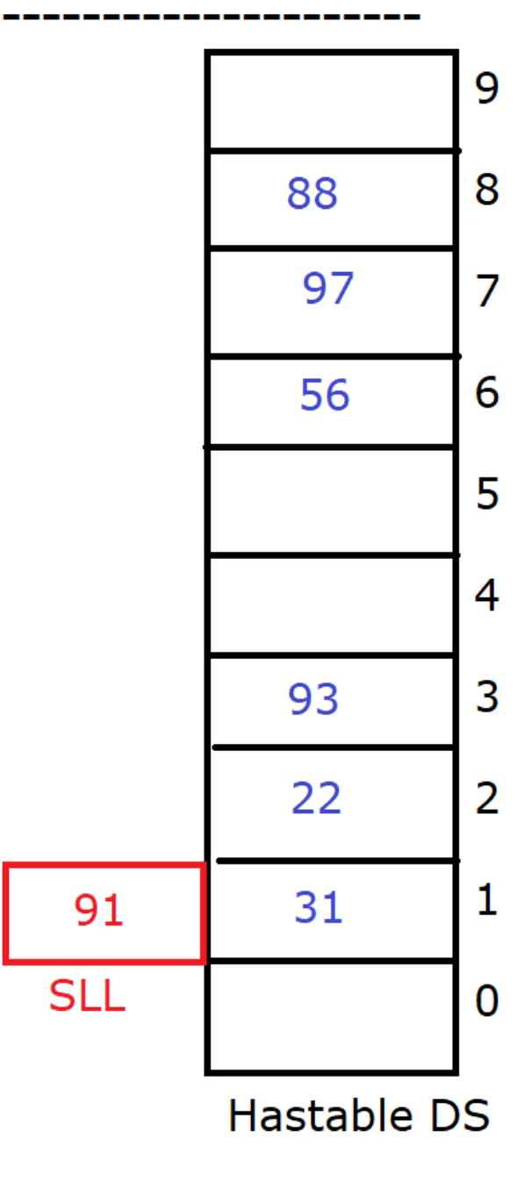
93 % 10 = 3

....

Now, we want to insert one more element i.e. 91

91 % 10 = 1

The 91 object element will insert in Bucket number 1, but in the bucket number 1 already we have an object i.e. 31 so, It is **Hash Collision,** In order to resolve this issue we should use Singly Linked List



HashSet<E> [Un-Ordered, Un-Sorted and No duplicates]

public class HashSet<E> extends AbstractSet<E> implements Set<E>, Cloneable, Serializable

* It is a predefined class available in java.util package from JDK 1.2V

* It can accept null, homogeneous and hetreogeneous objects.

* It does not accept duplicate objects [hashCode() & equals(Object obj) methods contract]

* It is an unordered & un-sorted set.

* When we iterate the elements through HashSet then the order will be un-predictable.

* It uses **Hashtable data structure.**

*** In order the insert the object in the Hashtable data structure, it uses hash code of the object.**

* Default capacity is 16 so, initially 16 buckets will be created internally.

* Methods are not synchronized.

* Iterator is fail fast iterator.

* It is mainly used to provide **fast searching operation.**

* It provides constant time performance for basic operation like add, delete & search.

Constructors :

* We have 4 types of constructors in the HashSet class.

1) HashSet hs1 = new HashSet();
It will create the HashSet Object with default capacity is 16 (Internally HashMap object). The default load fator or Fill Ratio is 0.75   (75% of HashSet is filled up then new HashSet Object will be created having double capacity and elements are inserted by using Re-Hashing)

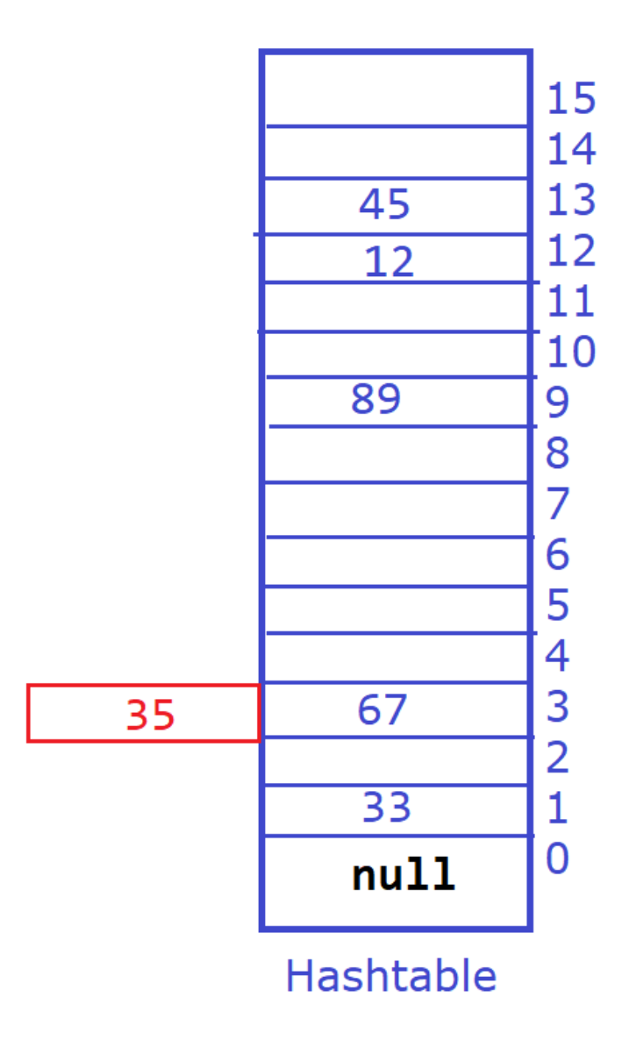
2) HashSet hs2 = new HashSet(int initialCapacity);
will create the HashSet object with user specified capacity.

3) HashSet hs3 = new HashSet(int initialCapacity, float loadFactor);
we can specify our own initialCapacity and loadFactor(by default load factor is 0.75)

4) HashSet hs4 = new HashSet(Collection c);
Interconversion of Collection.

How HashSet<E> works internally ?

```
 Set<Integer> hs = new HashSet<>();
         hs.add(67);
        hs.add(89);
        hs.add(33);
        hs.add(45);
        hs.add(12);
        hs.add(35);
        hs.add(null);      0 % 16 = 0
        67 % 16 = 3
        89 % 16 = 9
        -----------
Output : Bottom to top & right to left
        [33, 67, 35, 89, 12, 45]
```



```
Few Challanges they faced in this application :
-------------------------------------------------
1) 35 % 16 = 3
   12 % 16 = 12
   Java % 16 ???
   true % 16 ???
   emp % 16 ???
   We must have some kind of numeric value for all the objects : hashCode()

   "Ravi".hashCode() % 16 =  Bucket Index

2)How to deal with null
  null.hashCode()  -> NPE
  null has default hash code value -> 0


3)  In hashtable, elements are inserted randomly so they provided Load Factor OR Fill Ratio

   By default, default capacity is 16 & load factor is 0.75 that means 75% of Hashtable
   will be filled up then only new Hashtable data structure will be created.
```

//Programs :

```java
 package com.ravi.hash_set_demo;

 import java.util.HashSet;
 import java.util.Set;

 public class HashSetDemo1
 {
  public static void main(String args[])
  {
        Set<Integer> hs = new HashSet<>();

        hs.add(67);
       hs.add(89);
       hs.add(33);
       hs.add(45);
       hs.add(12);
       hs.add(35);
       hs.add(null);

       hs.forEach(num-> System.out.println(num));
    }
 }
```

```java
package com.ravi.hash_set_demo;

import java.util.HashSet;

public class HashSetDemo2
 {
    public static void main(String[] argv)
    {
        HashSet<String> hs = new HashSet<>();
        hs.add("Ravi");
        hs.add("Vijay");
        hs.add("Ravi");
        hs.add("Ajay");
        hs.add("Palavi");
        hs.add("Sweta");
        hs.add(null);
        hs.add(null);
        hs.forEach(str -> System.out.println(str));

    }
}
```

Output is un-predictable

```java
package com.ravi.hash_set_demo;

import java.util.Arrays;
import java.util.HashSet;


public class HashSetDemo3
{
    public static void main(String[] args)
    {
        Boolean []bool = new Boolean[5];
        HashSet<Object> set = new HashSet<>();
        bool[0] =  set.add(12);
        bool[1] =  set.add(12);
        bool[2] =  set.add("Java");
        bool[3] =  set.add(new String("Java"));
        bool[4] =  set.add("Nit");

        IO.println(Arrays.toString(bool));

        set.forEach(IO::println);

    }
}
```

```java
  package com.ravi.hash_set_demo;

  //add, delete, display and exit

  import java.util.HashSet;
  import java.util.Scanner;

  public class HashSetDemo4
  {
      public static void main(String[] args)
      {
    HashSet<String> hashSet = new HashSet<>();

    while (true)
      {
       System.out.println("Options:");
       System.out.println("1. Add element");
       System.out.println("2. Delete element");
       System.out.println("3. Display HashSet");
       System.out.println("4. Exit");

       System.out.print("Enter your choice (1/2/3/4): ");
       int choice = Integer.parseInt(IO.readln());


       switch(choice)
       {
           case 1->
           {
              String element = IO.readln("Enter the element to add ");
              if(hashSet.add(element))
              {
                  IO.println(element+" added successfully");
              }
              else
              {
                  IO.println("Duplicate element is not allowed");
              }
           }

           case 2 ->
           {
              String element = IO.readln("Enter the element to delete ");

              boolean isRemoved = hashSet.remove(element);
              if(isRemoved)
              {
                  IO.println(element+ " deleted successfully");
              }
              else
              {
                  IO.println(element+" is not available to delete");
              }
           }

           case 3->
           {
              hashSet.forEach(ele -> IO.println(ele));

           }

           case 4->
           {
              IO.println("Exiting from application");
              System.exit(0);
           }
       }

      }
     }
  }
```

```java
package com.ravi.hash_set_demo;

import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class HashSetDemo5
{
     public static void main(String[] args)
     {
       Set<String> setString = new HashSet<>();
       setString.add(new String("Java"));
       setString.add(new String("Java"));
       IO.println("Size of String is :"+setString.size());  //1


       IO.println("..............");
       Set<StringBuffer> setBuffer = new HashSet<>();
       setBuffer.add(new StringBuffer("Java"));
       setBuffer.add(new StringBuffer("Java"));
       IO.println("Size of StringBuffer is :"+setBuffer.size()); //2


     }
}



void main()
{
 String s1 = new String("Java");
 String s2 = new String("Java");
 IO.println(s1.equals(s2));
 IO.println(s1.hashCode()+" : "+s2.hashCode());


 StringBuffer sb1 = new StringBuffer("HTML");
 StringBuffer sb2 = new StringBuffer("HTML");
 IO.println(sb1.equals(sb2));
 IO.println(sb1.hashCode()+" : "+sb2.hashCode());
}
```